

Service Connections

Why Service Connections exist (core concept)

A **Service Connection** in Azure DevOps is:

"A saved, secure identity that allows Azure DevOps pipelines to talk to external systems."

Pipelines are **not you**.

They need **their own identity + permissions**.

So instead of:

```
az login  
kubectl apply  
docker push
```

The pipeline does it **automatically**, using service connections.

1 Azure Resource Manager service connection

Scope: Subscription

What this allows:

Azure DevOps → **Azure APIs**

Used for:

- Creating/updating Azure resources
- Talking to AKS control plane
- Managing identities & permissions
- (Sometimes) Helm deployments

In THIS project:

✓ Allows pipelines to:

- Authenticate to Azure
- Access AKS cluster metadata
- Work with managed identities

Think of it as:

| "Pipeline's Azure login"

2 Kubernetes service connection

Point to AKS

What this allows:

Azure DevOps → **Kubernetes API** (`kubectl`)

Used for:

- `kubectl apply`
- `kubectl rollout`
- `kubectl get pods`
- Helm installs/upgrades

In THIS project:

✓ Deploys the voting app to AKS:

- frontend
- backend
- postgres
- services
- ingress

Without this:

- Pipeline cannot touch Kubernetes
- Deployments would be manual

Think of it as:

| "Pipeline's kubectl access"

3 Azure Container Registry service connection

Select `votingacr123`

What this allows:

Azure DevOps → **ACR (Docker registry)**

Used for:

- `docker build`
- `docker push`
- `az acr login`

In THIS project:

✓ Builds images from source code:

- vote app
- worker
- result app

✓ Pushes them to:

```
votingacr123.azurecr.io
```

Think of it as:

| "Pipeline's Docker credentials"

How this all fits together

When you push code to Git:

1 ACR Service Connection

- Builds Docker images
- Pushes them to `votingacr123`

2 Azure Resource Manager Connection

- Authenticates to Azure
- Gets AKS details
- Confirms permissions

3 Kubernetes Service Connection

- Pulls new images from ACR
- Deploys them to AKS
- Updates running pods

👉 No passwords

👉 No manual kubectl

👉 No docker login

Fully automated

Why this is done THIS way (real-world reason)

In production:

Actor	What they do
Developer	Push code
Azure DevOps	Build + deploy
AKS	Runs workloads

Actor	What they do
Key Vault	Stores secrets
Humans	Almost nothing

This separation:

- Improves security
- Enables auditing
- Scales teams
- Prevents “it works on my laptop” syndrome

Why THREE service connections? (common confusion)

Because **one connection ≠ all access**:

Connection	Talks to	Purpose
ARM	Azure	Infrastructure
Kubernetes	AKS API	Deploy workloads
ACR	Docker Registry	Images

Each has **least privilege**.

Create Service Connections

We will create **THREE** service connections:

1. Azure Resource Manager (Subscription)
2. Kubernetes (AKS)
3. Azure Container Registry (ACR)

All from the **same place** in Azure DevOps.

Navigation

1. Open **Azure DevOps**
2. Select your **Organization**
3. Open your **Project**
4. Bottom-left → **Project settings**
5. Under **Pipelines** → **Service connections**
6. Click **New service connection**

We'll repeat this 3 times.

Azure Resource Manager service connection

This lets pipelines **log in to Azure**.

Click path

New serviceconnection → Azure Resource Manager

Choose authentication method

- ☒ **Service principal (automatic)**

Click **Next**

Configure it

Field	Value
Scope level	Subscription
Subscription	Select your Azure subscription
Resource Group	Leave empty
Service connection name	azure-subscription-sc
Description	(optional) Azure access for pipelines
Grant access permission	☒ Checked

Click **Save**

☒ Done.

Kubernetes service connection (AKS)

This lets pipelines run **kubectl / Helm**.

Click path

New serviceconnection → Kubernetes

Choose connection type

- ☒ Azure Kubernetes Service

Click **Next**

Configure it

Field	Value
Subscription	Select your subscription
Cluster	aks-voting
Namespace	Leave empty (all namespaces)
Service connection name	aks-voting-sc
Grant access permission	☒ Checked

Click **Save**

☒ Done.

Azure Container Registry service connection

This lets pipelines **docker build & push**.

Click path

New serviceconnection → Docker Registry

Choose registry type

- ☒ **Azure Container Registry**

Click **Next**

Configure it

Field	Value
Subscription	Select your subscription
Azure Container Registry	votingacr123
Service connection name	acr-voting-sc
Grant access permission	☒ Checked

Click **Save**

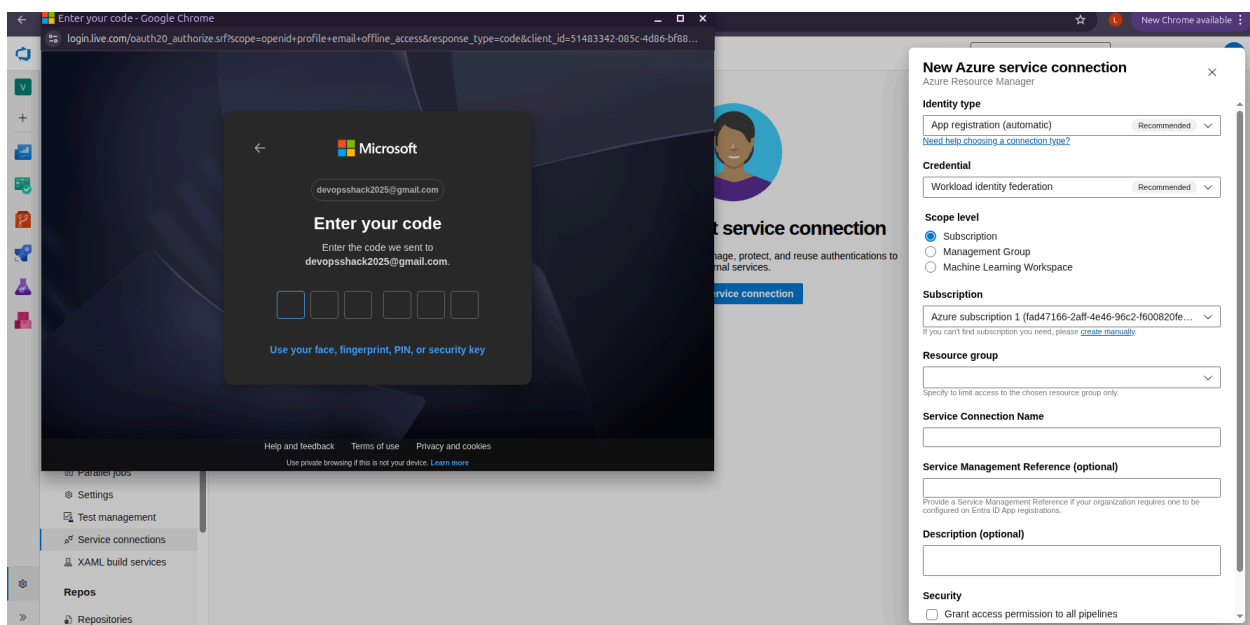
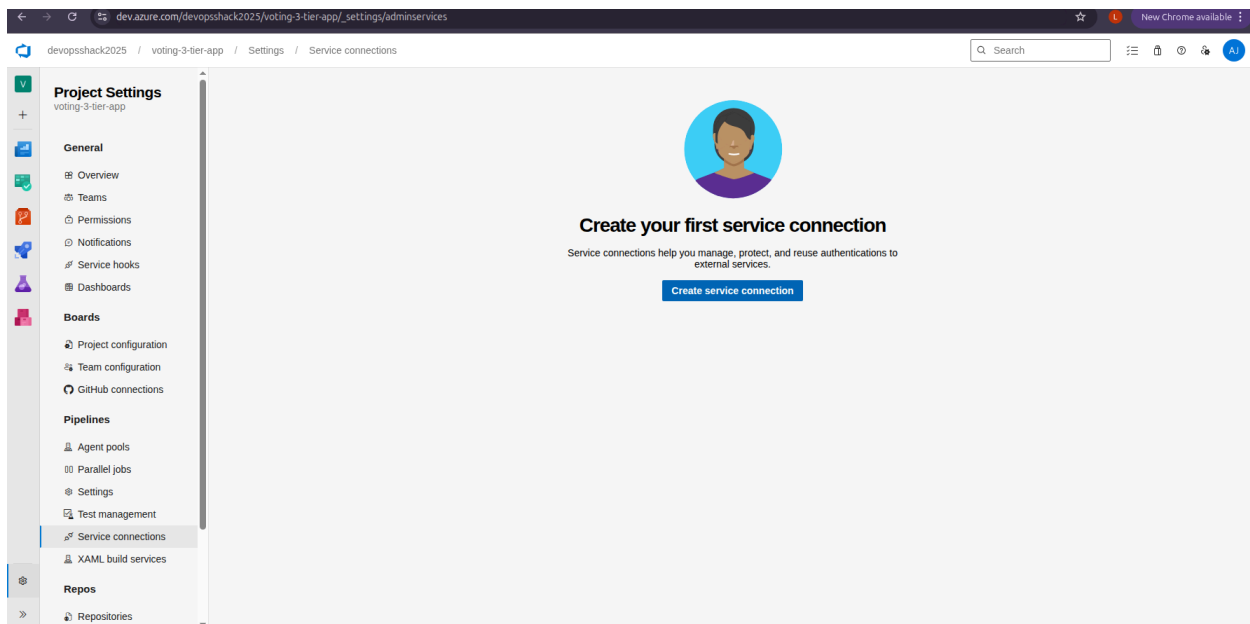
☒ Done.

Final verification

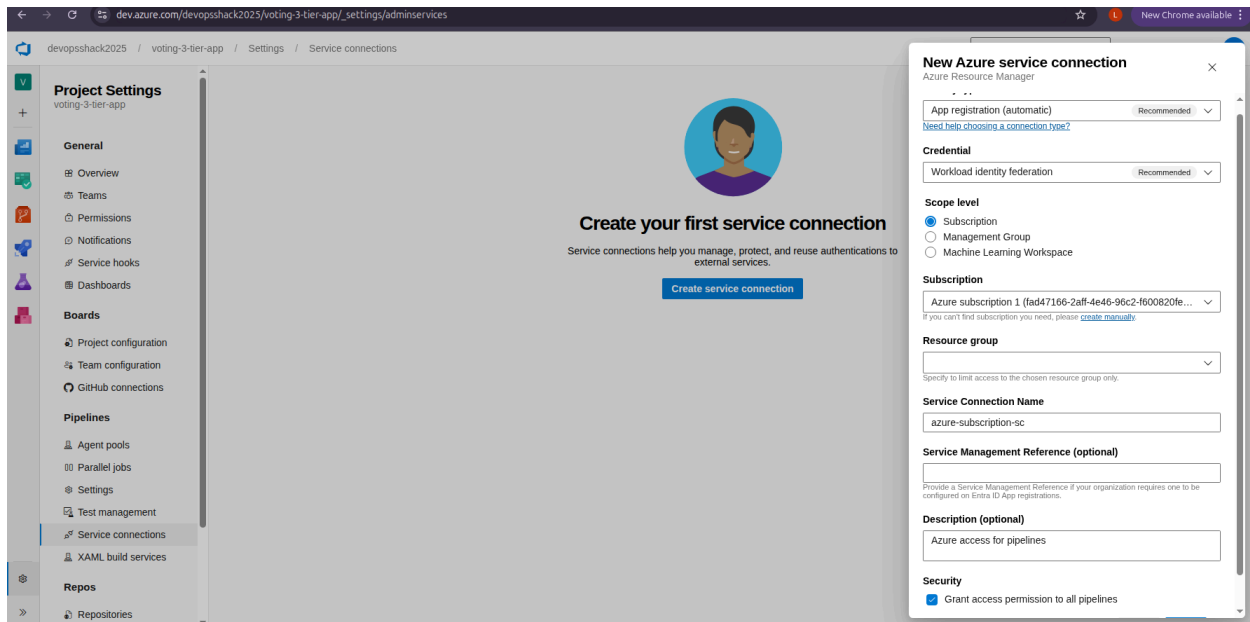
You should now see **three service connections**:

Name	Type
azure-subscription-sc	Azure Resource Manager
aks-voting-sc	Kubernetes
acr-voting-sc	Docker Registry

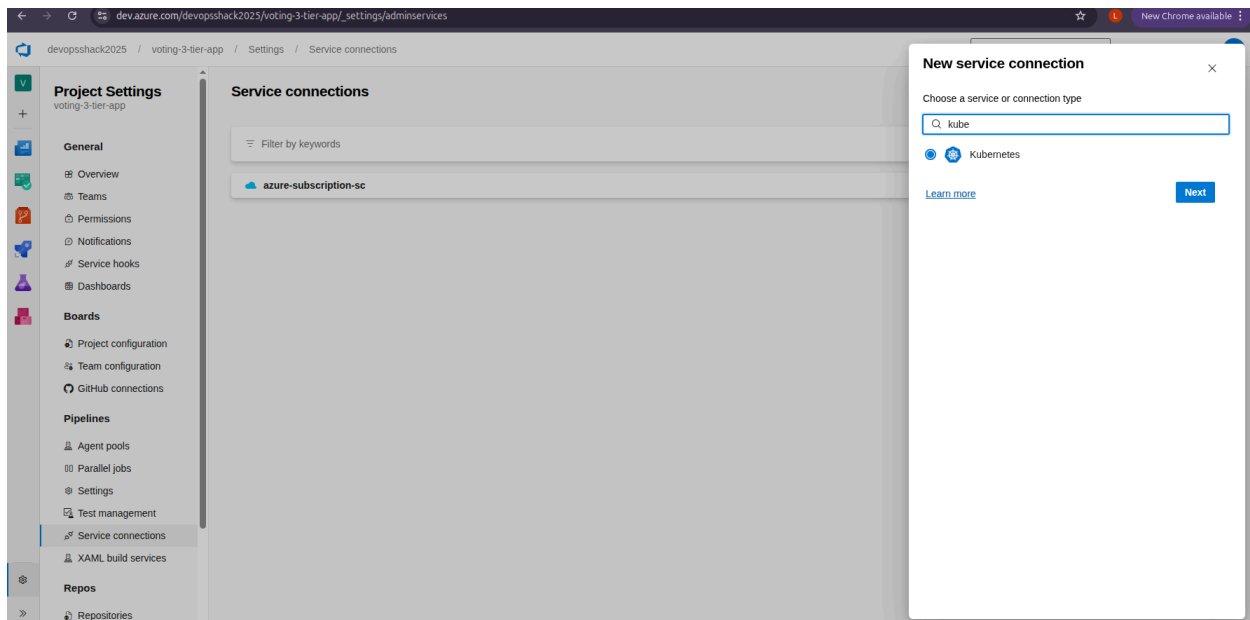
If even **one is missing**, pipelines will fail later.



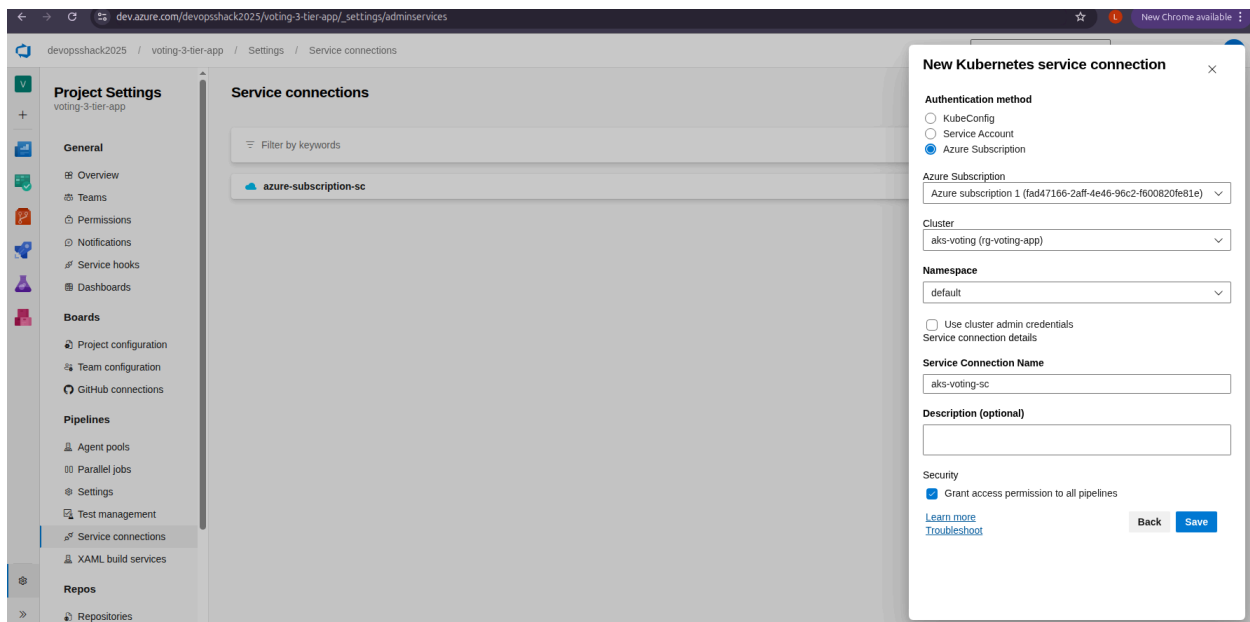
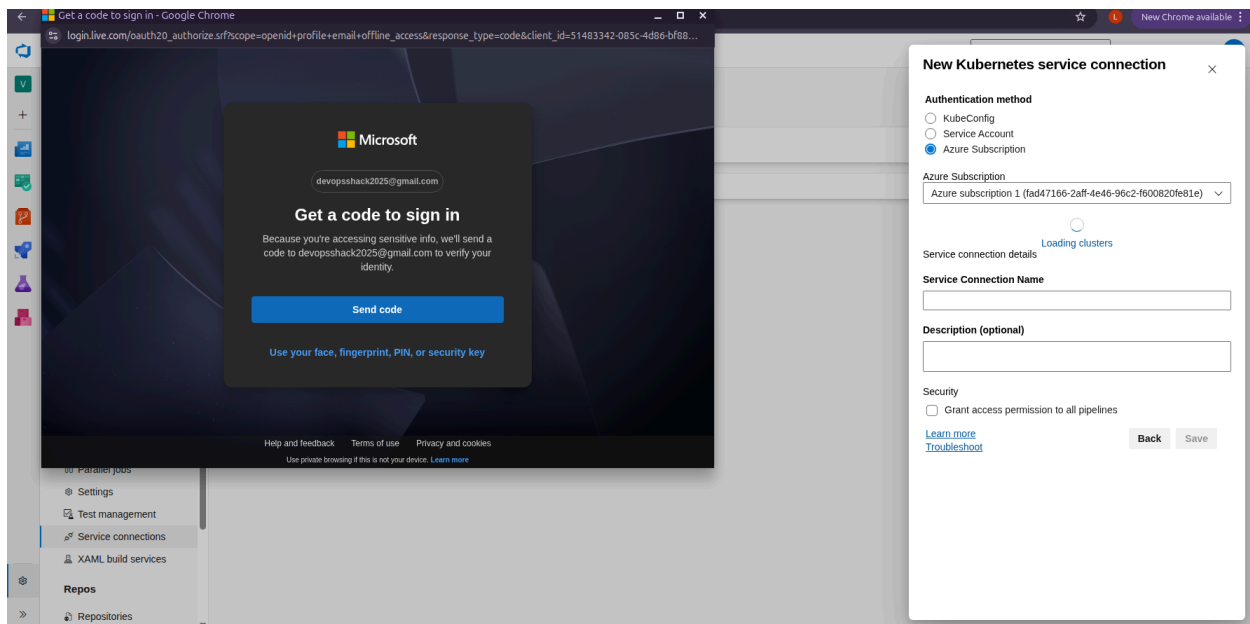
Save with the below settings:



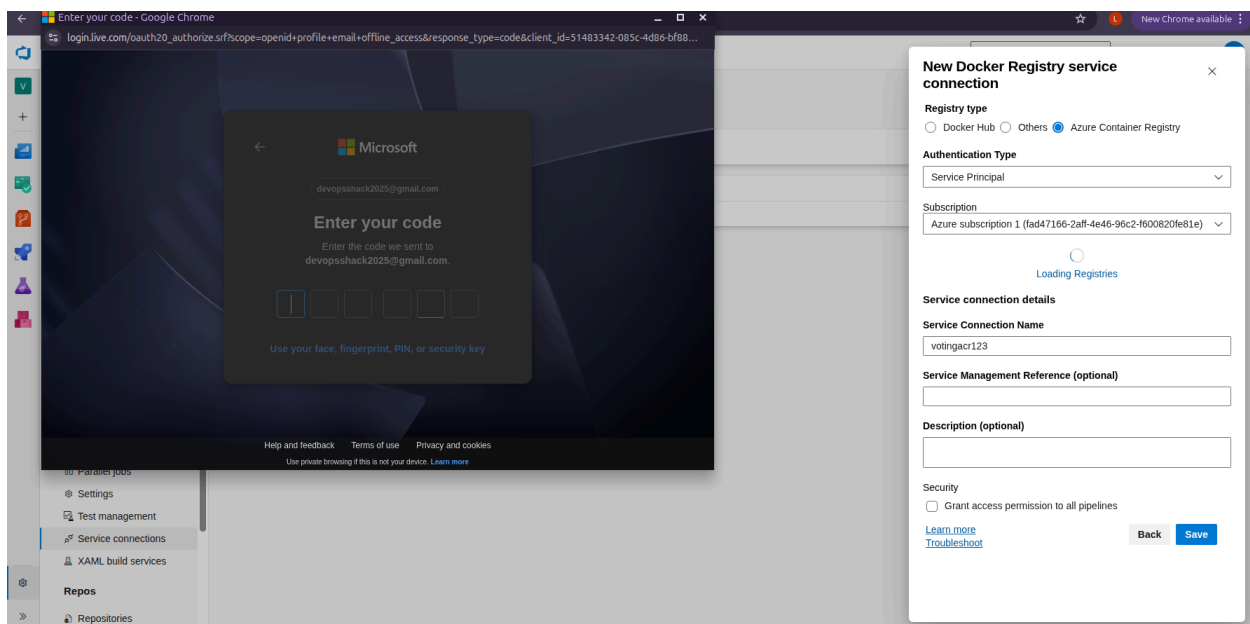
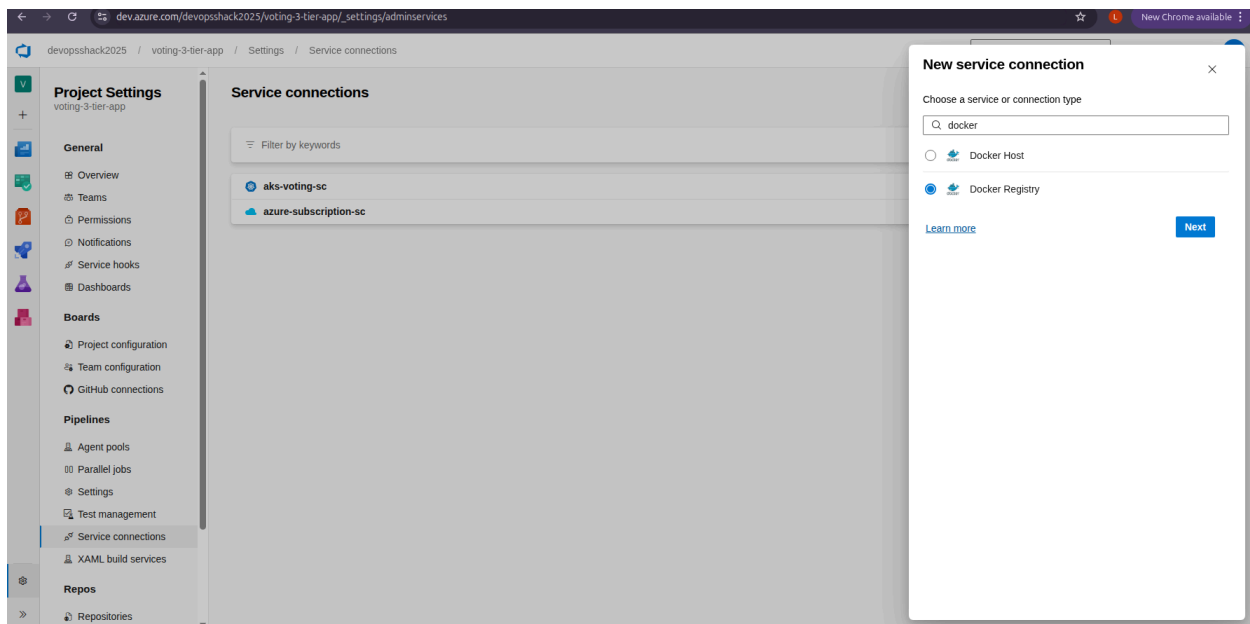
Kubernetes Connection

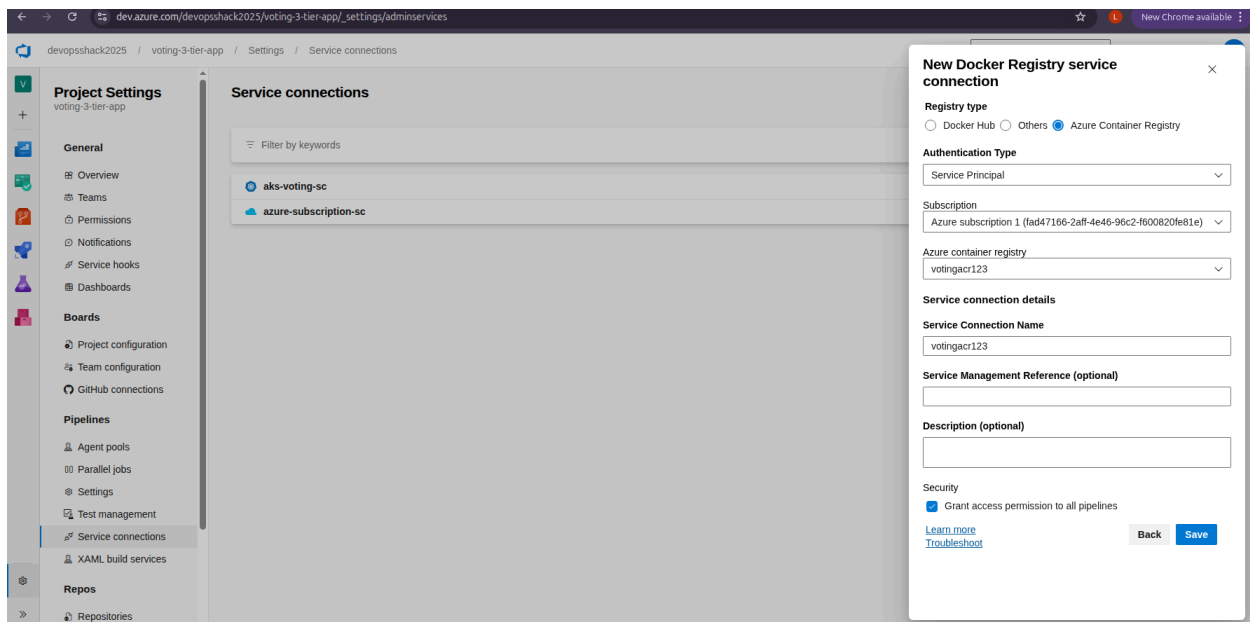


Authenticate again



Docker Registry Connection





You should have all 3 service connections ready.